

1. Instance

- **Definition:** An **instance** refers to the actual data stored in the database at a specific moment in time. It represents the current state of the database, including all the records, rows, and values in the tables.
- **Key Points:**
 - An instance is dynamic and changes over time as data is inserted, updated, or deleted.
 - It is a snapshot of the database at a particular point in time.
 - The schema (structure) remains the same, but the data within it (the instance) can change.
- **Example:**
 - Consider a Student table with the following schema:

```
Student (Student_ID, Name, DOB, Department_ID)
```

At a given time, the instance of the Student table might look like this:

Student_ID	Name	DOB	Department_ID
101	John Doe	2000-01-15	1
102	Jane Smith	2001-05-20	2

- If a new student is added, the instance changes, but the schema remains the same.

2. Data Independence

- **Data independence** refers to the ability to modify the schema at one level of the database system without affecting the schema at the next higher level. It ensures that changes in the database structure do not impact the applications or users accessing the data.
- - **Example:**
 - Adding a new column to a table.
 - Splitting a table into two tables.

- These changes do not require rewriting the views or queries used by applications.

Example of Data Independence

- **Scenario:**
 - A Student table has the following schema:

```
Student (Student_ID, Name, DOB, Department_ID)
```

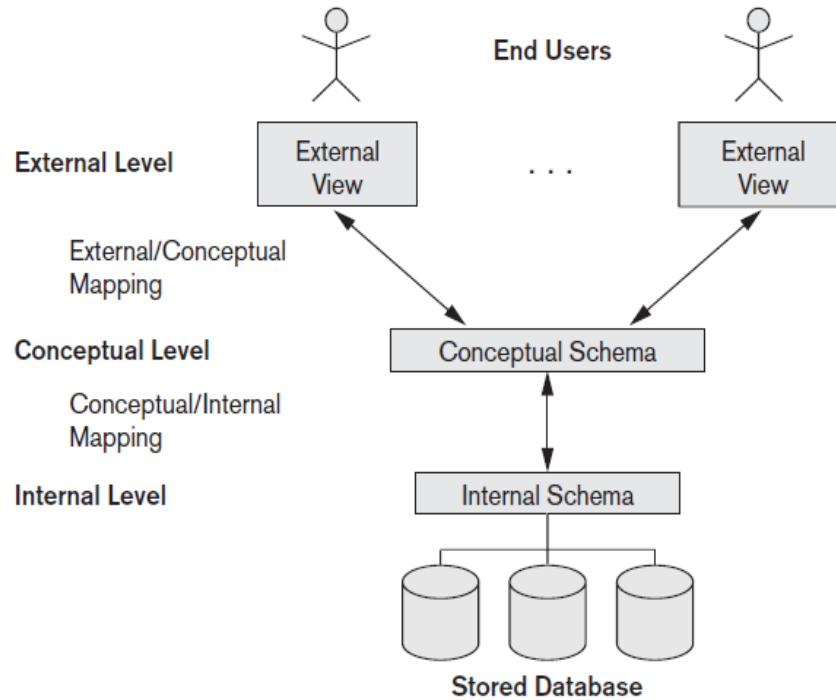
- An application uses this table to display student names and DOBs.
 - A new column, Email, is added to the Student table.
 - The application continues to work without any changes because it does not use the new column.

Three-Tier Schema Architecture

The **Three-Tier Schema Architecture** in DBMS is a framework that helps achieve **data independence**, meaning changes in one level of the database do not affect the other levels. This architecture separates the database into three layers:

1. **External Level (View Level)**
2. **Conceptual Level (Logical Level)**
3. **Internal Level (Physical Level)**

Figure 2.2
The three-schema architecture.



1. External Level (View Level)

- This is the **top layer** where users interact with the database.
- It provides a customized view of the database for different users or applications.
- Each user sees only the data they need, and irrelevant data is hidden.

Example:

Imagine a **school database**.

- A **student** logs in and sees only their grades, attendance, and personal details.
- A **teacher** sees the list of students, their grades, and course details.
- The **admin** sees everything, including financial data and staff details.

2. Conceptual Level (Logical Level)

- This is the **middle layer** that defines the overall structure of the database.
- It describes what data is stored, how it is related, and the constraints (rules) applied to the data.
- It acts as a bridge between the external and internal levels.

Example:

In the school database, the conceptual level defines:

- Tables like **Students**, **Teachers**, **Courses**, and **Grades**.
- Relationships like "a student enrolls in a course" or "a teacher teaches a course."
- Rules like "a student cannot have more than 5 courses."

3. Internal Level (Physical Level)

- This is the **bottom layer** that deals with how the data is physically stored in the database.
- It includes details like file storage, indexing, compression, and data access methods.

Example:

In the school database, the internal level defines:

- How the **Students** table is stored on the hard disk (e.g., in a file called students.dat).
- Whether the data is stored in a **B-tree** or **hash index** for faster searching.
- How data is compressed to save space.

This level is hidden from users and is managed by the database administrators.

EXAMPLE:**1. External Level:**

- A **customer** sees product details, prices, and order history.
- A **seller** sees inventory, sales, and customer reviews.
- A **manager** sees sales reports, profits, and employee performance.

2. Conceptual Level:

- Defines tables like **Products**, **Orders**, **Customers**, and **Sellers**.
- Establishes relationships like "a customer places an order" or "a seller lists a product."

3. Internal Level:

- Stores data in files like products.dat and orders.dat.

Database System Structure

A database system is made up of several components that work together to store, manage, and retrieve data efficiently. The main components are:

1. **Database:** The actual storage of data (e.g., tables, files).
2. **DBMS (Database Management System):** Software that manages the database (e.g., MySQL, Oracle).
3. **Users:** People or applications that interact with the database (e.g., students, teachers, apps).
4. **Applications:** Programs that use the database (e.g., a school management system).
5. **Hardware:** The physical devices like servers, storage, and computers.

Example:

Think of a **library system**:

- The **database** is the collection of books and their details (title, author, etc.).
- The **DBMS** is the librarian who manages the books (adds, removes, or finds books).
- The **users** are students or teachers who borrow or return books.
- The **application** is the software used to search for books or issue them.
- The **hardware** is the library building, shelves, and computers.

Database Environment

The database environment refers to the setup where the database system operates. It includes:

1. **Data:** The actual information stored (e.g., student records, product details).
2. **Software:** DBMS and applications that interact with the database.
3. **Hardware:** Servers, storage devices, and networking equipment.
4. **Procedures:** Rules for using the database (e.g., how to add or delete data).
5. **People:** Database administrators (DBAs), developers, and end-users.

Real-Time Example:

In a **hospital management system**:

- **Data:** Patient records, doctor details, appointments.
- **Software:** The hospital management software (e.g., Cerner).
- **Hardware:** Servers storing patient data and computers used by staff.
- **Procedures:** Rules for updating patient records or scheduling appointments.
- **People:** Doctors, nurses, IT staff, and patients.

Centralized Database Architecture

In a **centralized architecture**, all the data is stored and managed in a single location (one server). Users access the database from this central location.

Example:

A small bank may store all customer and transaction records on one central server in its data centre.

Advantages:

- Easy to manage and maintain.
- Low cost for small setups.

Disadvantages:

- Single point of failure (if the server crashes, everything stops).
- Limited scalability (cannot handle too many users or large amounts of data).

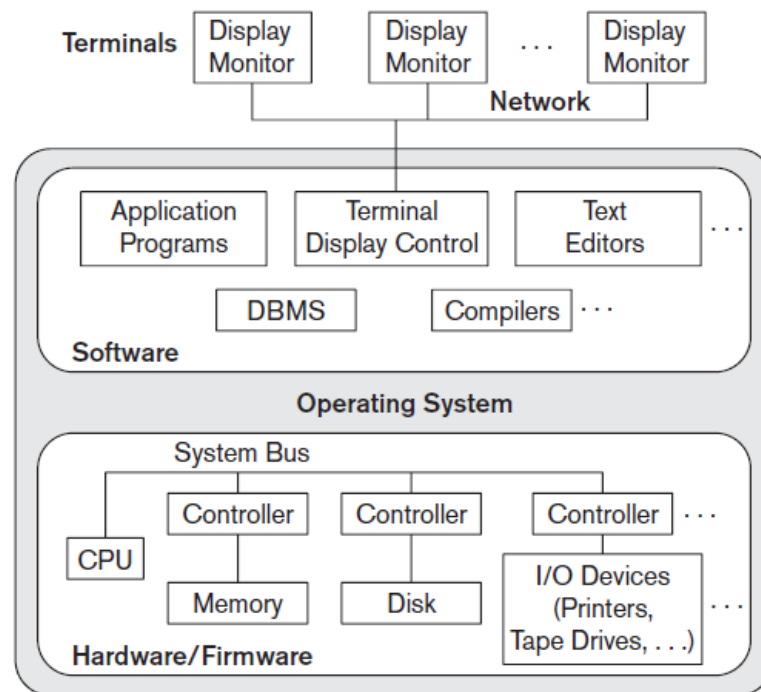


Figure 2.4
A physical centralized architecture.

Client-Server Database Architecture

In a **client-server architecture**, the database is stored on a **server**, and users (clients) access it over a network. The server handles requests from multiple clients.

Example:

Think of an **online shopping platform** like Amazon:

- The **server** stores all the product details, customer orders, and payment information.
- The **clients** are users accessing the platform through their laptops, phones, or tablets.
- When you search for a product, your device (client) sends a request to the server, which processes it and sends back the results.

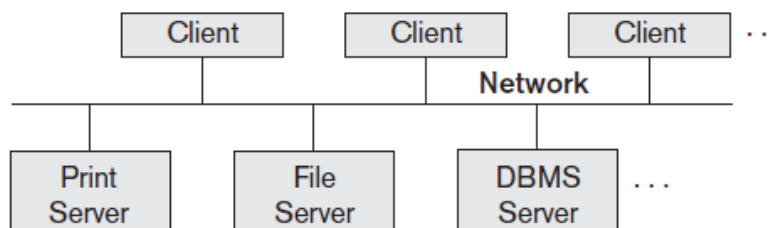


Figure 2.5
Logical two-tier client/server architecture.

Advantages:

- Scalable (can handle many users and large amounts of data).
- No single point of failure (backup servers can be used).
- Efficient for distributed environments.

Disadvantages:

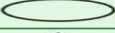
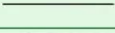
- More complex to set up and maintain.
- Requires good network infrastructure.

Comparison: Centralized vs. Client-Server Architecture

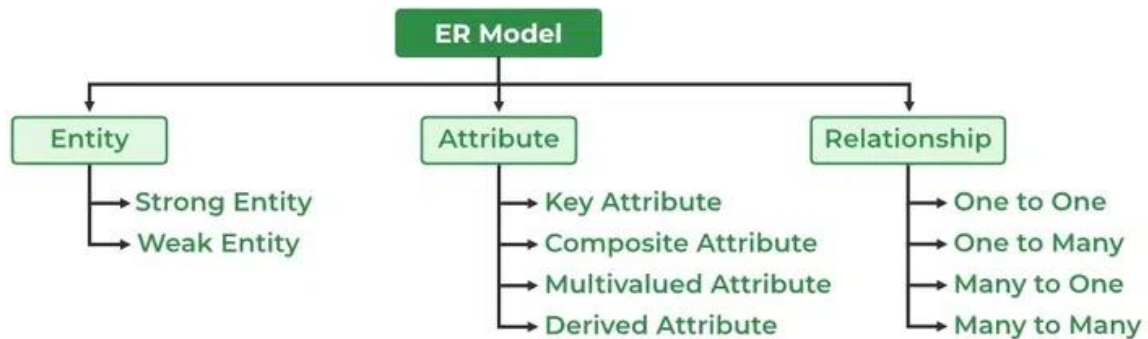
Feature	Centralized Architecture	Client-Server Architecture
Data Location	Single server	Central server, accessed by clients over a network
Scalability	Limited	Highly scalable
Cost	Low (for small setups)	Higher (due to network and server costs)
Failure Risk	Single point of failure	Distributed, less risk of failure
Example	Small retail store	Online shopping platform

Entity Relationship Model:

- ER model stands for an Entity-Relationship model. This model is used to define the data elements and relationship for a specified system.
- Symbols Used in ER Model
-

Figures	Symbols	Represents
Rectangle		Entities in ER Model
Ellipse		Attributes in ER Model
Diamond		Relationships among Entities
Line		Attributes to Entities and Entity Sets with Other Relationship Types
Double Ellipse		Multi-Valued Attributes
Double Rectangle		Weak Entity

Components of ER Diagram



Entity

An Entity may be an object with a physical existence – a particular person, car, house, or employee – or it may be an object with a conceptual existence – a company, a job, or a university course.

Entity Set

An **Entity Set** is a collection of similar entities that belong to the same **entity type** in a database or data model.

Example:

🔗 **Entity Type:** Student

🔗 **Entity Set:** All the students in the system

Student_ID	Name	Age	Grade	Contact Number
101	Rahul K	10	5	9876543210
102	Priya M	11	6	9876512345
103	Arjun N	12	7	9876587654

In this table:

- **Entity Type:** Student

- **Entity Set:** {Rahul K, Priya M, Arjun N}

Types of Entity

1. Strong Entity

A [Strong Entity](#) is a type of entity that has a key Attribute. Strong Entity does not depend on other Entity in the Schema. It has a primary key, that helps in identifying it uniquely, and it is represented by a rectangle. These are called Strong Entity Types.

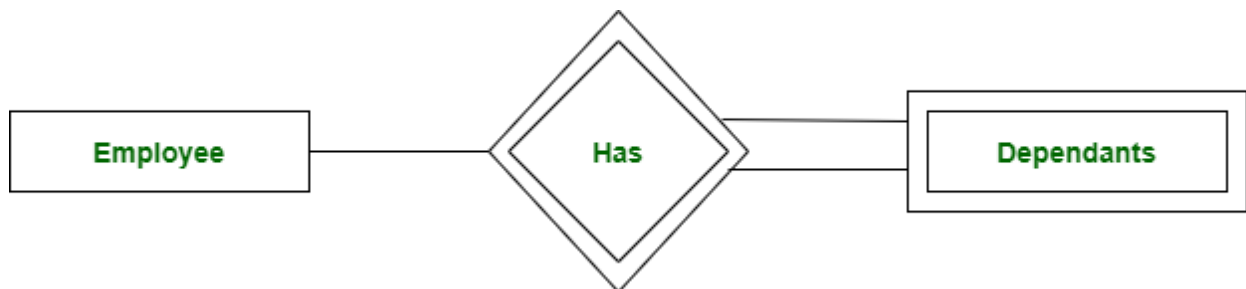
2. Weak Entity

A **Weak Entity** is an entity that **cannot be uniquely identified by its own attributes alone**. It depends on a **related strong entity (owner entity)** for its identification.

For Example, A company may store the information of dependents (Parents, Children, Spouse) of an Employee. But the dependents can't exist without the employee. So Dependent will be a **Weak Entity Type**.

A weak entity type is represented by a Double Rectangle.

The participation of weak entity types is always total. The relationship between the weak entity type and its identifying strong entity type is called identifying relationship and it is represented by a double diamond.



Attributes

[Attributes](#) are the properties that define the entity type. For example, Roll_No, Name, DOB, Age, Address, and Mobile_No are the attributes that define entity type Student. In ER diagram, the attribute is represented by an oval.



Types of Attributes

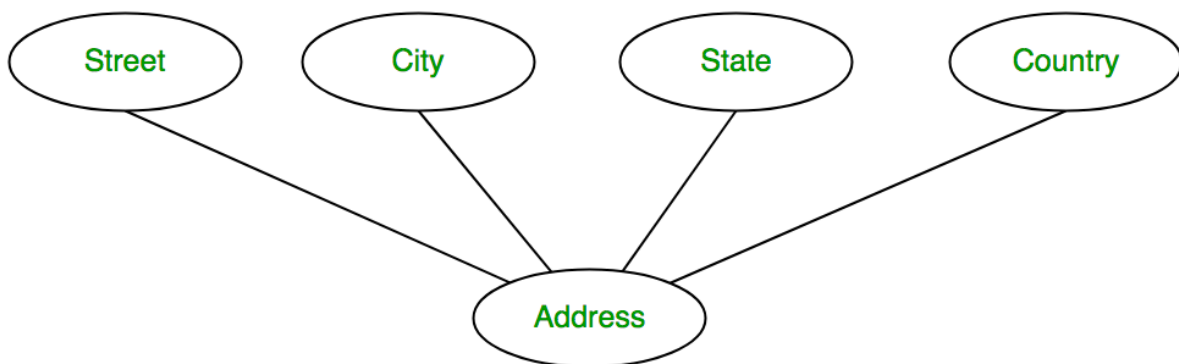
1. Key Attribute

The attribute which **uniquely identifies each entity** in the entity set is called the key attribute. For example, Roll_No will be unique for each student. In ER diagram, the key attribute is represented by an oval with underlying lines.



2. Composite Attribute

An attribute **composed of many other attributes** is called a composite attribute. For example, the Address attribute of the student Entity type consists of Street, City, State, and Country. In ER diagram, the composite attribute is represented by an oval comprising of ovals.



Multivalued Attribute

An attribute consisting of more than one value for a given entity. For example, Phone_No (can be more than one for a given student). In ER diagram, a multivalued attribute is represented by a double oval.

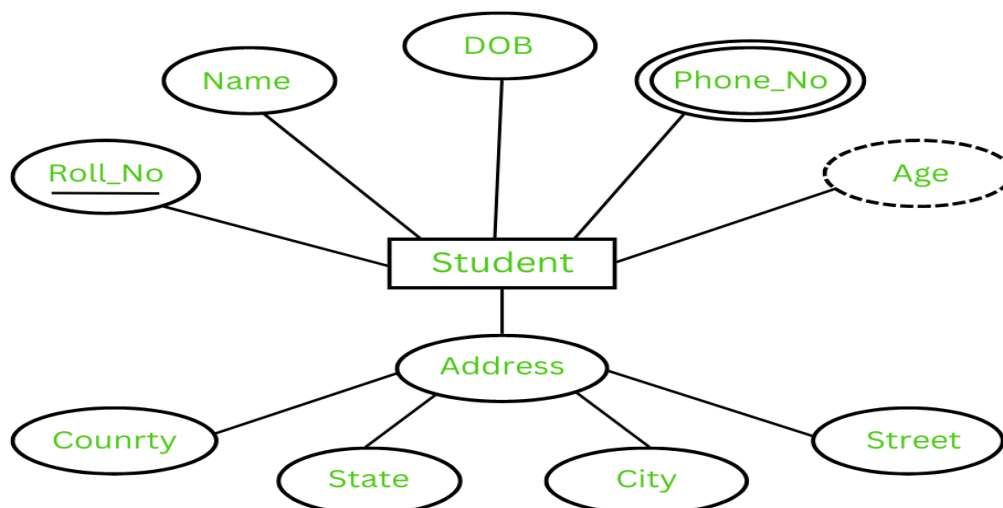


4. Derived Attribute

An attribute that can be derived from other attributes of the entity type is known as a derived attribute. e.g.; Age (can be derived from DOB). In ER diagram, the derived attribute is represented by a dashed oval.

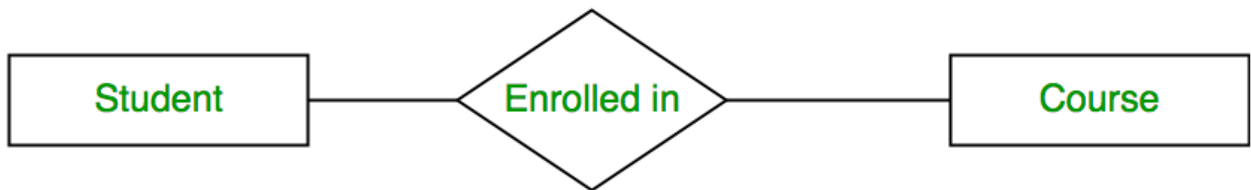


The Complete Entity Type Student with its Attributes can be represented as:

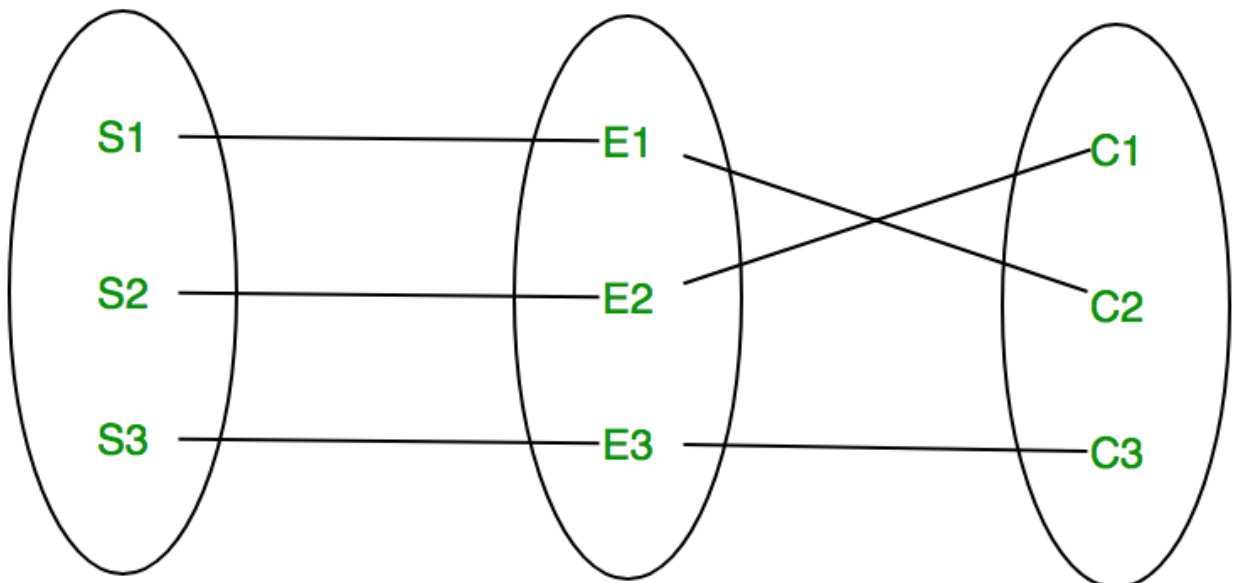


Relationship Type and Relationship Set

A Relationship Type represents the association between entity types. For example, 'Enrolled in' is a relationship type that exists between entity type Student and Course. In ER diagram, the relationship type is represented by a diamond and connecting the entities with lines.



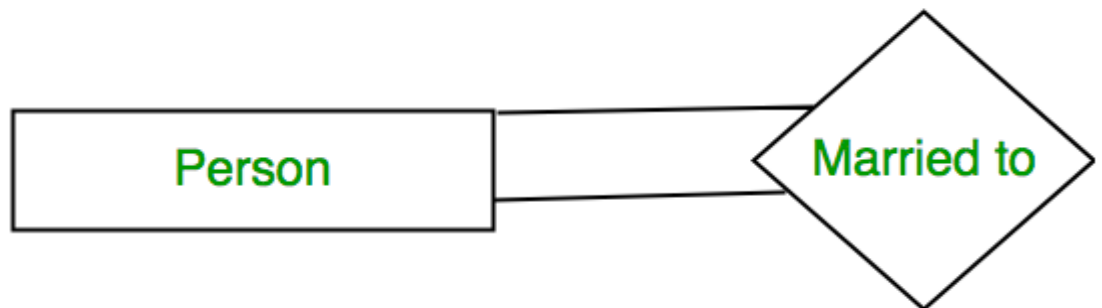
A set of relationships of the same type is known as a relationship set. The following relationship set depicts S1 as enrolled in C2, S2 as enrolled in C1, and S3 as registered in C3.



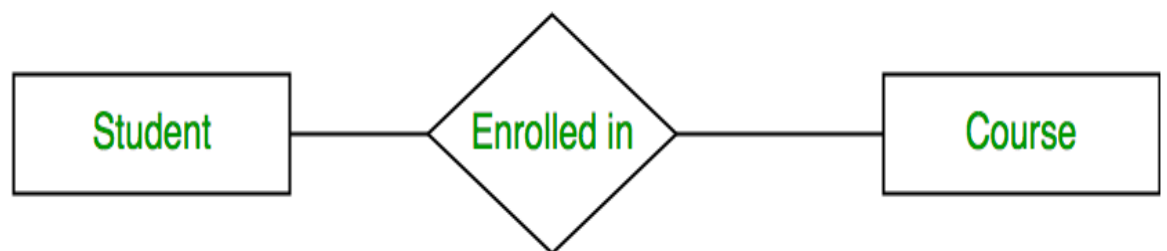
Degree of a Relationship Set

The number of different entity sets participating in a relationship set is called the [degree of a relationship set](#).

1. **Unary Relationship:** When there is only ONE entity set participating in a relation, the relationship is called a unary relationship. For example, one person is married to only one person.



2. **Binary Relationship:** When there are TWO entities set participating in a relationship, the relationship is called a binary relationship. For example, a Student is enrolled in a Course.



3. **Ternary Relationship:** When there are three entity sets participating in a relationship, the relationship is called a ternary relationship.

4. **N-ary Relationship:** When there are n entities set participating in a relationship, the relationship is called an n-ary relationship.

What is Cardinality?

The number of times an entity of an entity set participates in a relationship set is known as [cardinality](#). Cardinality can be of different types:

1. One-to-One: When each entity in each entity set can take part only once in the relationship, the cardinality is one-to-one. Let us assume that a male can marry one female and a female can marry one male. So the relationship will be one-to-one.

the total number of tables that can be used in this is 2.

Doctors.

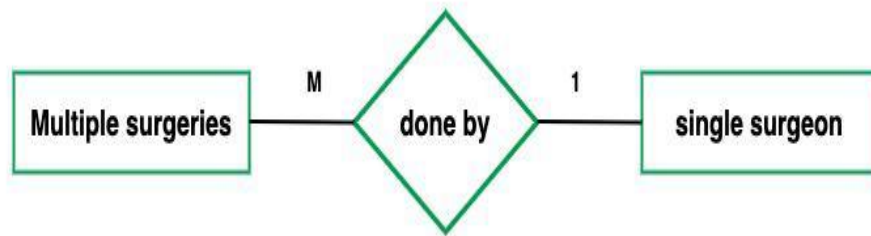


3. One-to-Many: In one-to-many mapping as well where each entity can be related to more than one entity and the total number of tables that can be used in this is 2. Let us assume that one surgeon department can accommodate many doctors. So the Cardinality will be 1 to M. It means one department has many



4. Many-to-One: When entities in one entity set can take part only once in the relationship set and entities in other entity sets can take part more than once in the relationship set, cardinality is many to one. Let us assume that a student can take only one course but one course can be taken by many students. So the cardinality will be n to 1. It

means that for one course there can be n students but for one student, there will be only one course.



- 5. Many-to-Many:** When entities in all entity sets can take part more than once in the relationship cardinality is many to many. Let us assume that a student can take more than one course and one course can be taken by many students. So the relationship will be many to many.

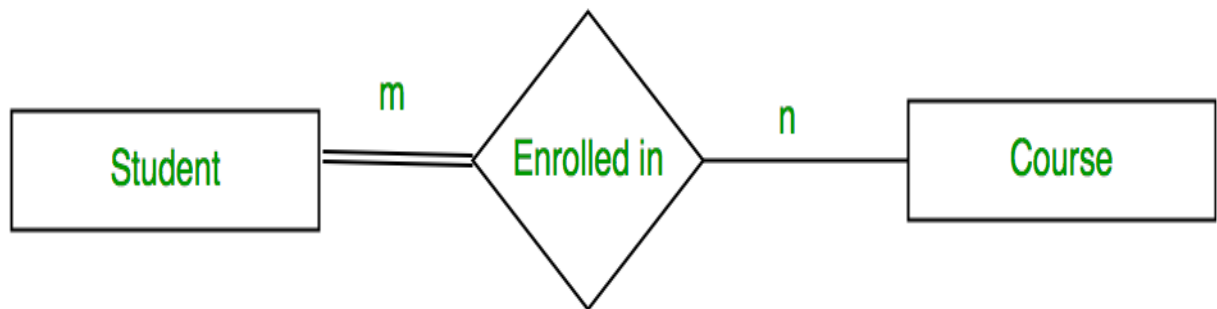


Participation Constraint

[Participation Constraint](#) is applied to the entity participating in the relationship set.

1. Total Participation – Each entity in the entity set must participate in the relationship. If each student must enroll in a course, the participation of students will be total. Total participation is shown by a double line in the ER diagram.

2. Partial Participation – The entity in the entity set may or may NOT participate in the relationship. If some courses are not enrolled by any of the students, the participation in the course will be partial.



Unit II

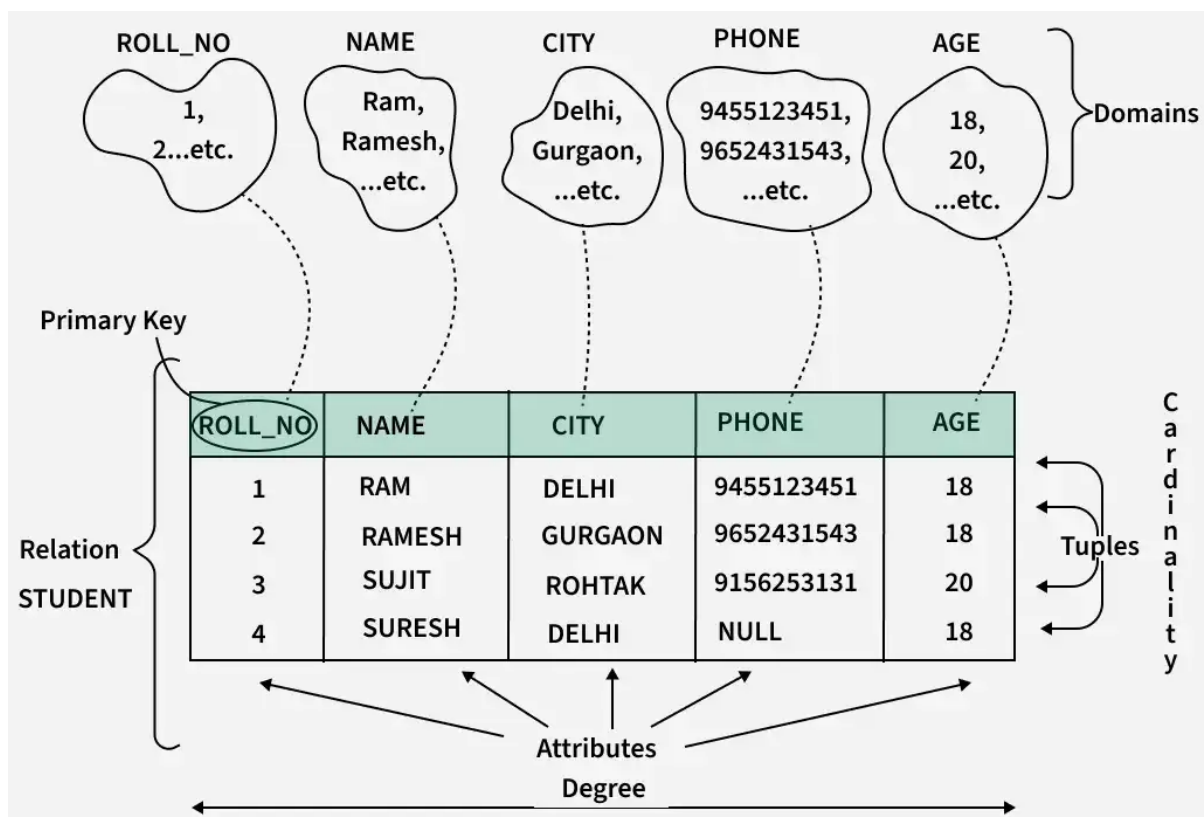
Syllabus:

Relational Model: Introduction to relational model, concepts of domain, attribute, tuple, relation, importance of null values, constraints (Domain, Key constraints, integrity constraints) and their importance, Relational Algebra, Relational Calculus.

Relational Model

The **Relational Model** represents data and their relationships through a collection of tables. Each table also known as a relation consists of rows and columns.

It was introduced by Edgar F. Codd in 1970



Key Terms

- **Degree:** The number of attributes in the relation is known as the degree of the relation. The STUDENT relation defined above has degree 5.
- **Cardinality:** The number of tuples in a relation is known as [cardinality](#). The STUDENT relation defined above has cardinality 4.
- **NULL Values:** The value which is not known or unavailable is called a NULL value. It is represented by NULL. e.g. PHONE of STUDENT having ROLL_NO 4 is NULL.

- **Relation Key:** These are basically the keys that are used to identify the rows uniquely or also help in identifying tables. These are of the following types:
 - [Primary Key](#)
 - [Candidate Key](#)
 - [Super Key](#)
 - [Foreign Key](#)
 - [Alternate Key](#)
 - [Composite Key](#)
 - [Primary Key](#): The primary key is an attribute or a set of attributes that help to uniquely identify the tuples(records) in the relational table.

Rules For Defining a Primary Key

- **Minimal:** The primary key is composed of a minimum number of attributes that satisfy the unique occurrence of the tuples or records.
- **NON NULL Value:** The primary key which refers to a nonnull value that is required for the identification of the tuple (record) means that any attribute can not contain the non-null value in [DBMS](#).
-
- [Candidate Key](#)

It refers to a set of one or more columns (attributes) in a table that can uniquely identify each row in that table.

2. Candidate Key

- A **Candidate Key** is a column or a set of columns that can uniquely identify each row but is **not yet chosen** as the primary key.
- A table can have **multiple candidate keys**, but only one can be chosen as the **Primary Key**.

StudentID	Email	Name	Age
101	a@gmail.com	Alice	22
102	b@yahoo.com	Bob	21

•

Example:

☐ StudentID and Email **both** are unique and can act as a **Candidate Key**.

☐ If StudentID is chosen as the **Primary Key**, then Email becomes an **Alternate Key**.

☐ StudentID and Email **both** are unique and can act as a **Candidate Key**.

☐ If StudentID is chosen as the **Primary Key**, then Email becomes an **Alternate Key**.

3. Super Key

- A **Super Key** is a set of one or more columns that can uniquely identify a row.
- A **Super Key** may have extra attributes that are **not necessary** for uniqueness.

Example:

For the **Students** table:

- {StudentID}
- {Email}
- {StudentID, Name}
- {StudentID, Email, Age}

✅ Note:

Every **Candidate Key** and **Primary Key** is a **Super Key**, but a **Super Key** may contain extra unnecessary attributes.

4. Composite Key

- A **Composite Key** is a key that consists of **two or more columns** to uniquely identify a record.
- Used when a **single column is not enough** to ensure uniqueness.

CREATE TABLE Enrollments (

StudentID INT,

CourseID INT,

EnrollmentDate DATE,

PRIMARY KEY (StudentID, CourseID)

);

Here, (StudentID, CourseID) together form a **Composite Key**, ensuring that a student cannot enroll in the same course twice.

5. Foreign Key

- A **Foreign Key** is a column in one table that references the **Primary Key** of another table.
- It helps maintain **referential integrity**.

CREATE TABLE Orders (

OrderID INT PRIMARY KEY,

StudentID INT,

```

OrderDate DATE,

FOREIGN KEY (StudentID) REFERENCES Students(StudentID)

);

```

Here, StudentID in **Orders** is a **Foreign Key** referencing StudentID in **Students**.

6. Alternate Key

- An **Alternate Key** is a **Candidate Key** that is **not chosen** as the **Primary Key**.

Example:

In the **Students** table:

- **Candidate Keys:** StudentID, Email
- If StudentID is chosen as the **Primary Key**, then Email becomes an **Alternate Key**.

Key Type	Definition	Example
Primary Key	Uniquely identifies rows; no NULLs or duplicates	StudentID in Students table
Foreign Key	Links two tables by referencing a primary key	StudentID in Courses references Students(StudentID)
Super Key	Any set of attributes that uniquely identifies rows	{StudentID}, {StudentID, FirstName}
Candidate Key	Minimal subset of attributes that uniquely identifies rows	{StudentID}, {SocialSecurityNumber}
Alternate Key	Candidate keys not chosen as the primary key	If StudentID is primary, then SocialSecurityNumber is alternate
Composite Key	Combination of columns used to uniquely identify rows	{StudentID, CourseID} in an Enrollments table

Concepts of domain:

In an **RDBMS (Relational Database Management System)**, a **domain** refers to the set of permissible values that a column (attribute) in a table can hold. It defines the data type, format, and constraints for the values in a specific column, ensuring data integrity and consistency.

- For example, a column representing "Age" may have a domain that restricts values to integers between 0 and 150.

• Domain Constraints:

- These are rules applied to ensure that only valid data is entered into a column. Examples include:
 - Data type (e.g., integer, string, date).

- Range restrictions (e.g., numbers between 1 and 100).
- Format restrictions (e.g., email addresses).

```
CREATE TABLE Employee (  
    EmployeeID INT NOT NULL,      -- Domain: Positive integers  
    Name VARCHAR(50) NOT NULL,   -- Domain: Strings up to 50 characters  
    Age INT CHECK (Age >= 18 AND Age <= 65), -- Domain: Integers between 18  
and 65  
    Email VARCHAR(100) UNIQUE     -- Domain: Valid email format  
);
```

- Here, each column has a specific domain:
 - EmployeeID: Must be a positive integer.
 - Name: Can only store strings up to 50 characters.
 - Age: Restricted to values between 18 and 65.
 - Email: Must be unique and follow an email format.

importance of null values

1. Representing Missing or Unknown Data

- Null values allow a database to handle situations where data is not available or not yet known. For example, if a customer's middle name is not provided, a null value can be used instead of leaving the field blank or using a placeholder (e.g., "N/A" or "Unknown"), which could lead to ambiguity.

2. Handling Inapplicable Data

- In some cases, certain fields may not be applicable to a specific record. For example, in a database storing employee information, the "termination date" field would not apply to

current employees. Using a null value in such cases is more appropriate than inserting a default or dummy value.

3. Flexibility in Data Modeling

- Null values provide flexibility in designing database schemas. They allow optional fields in tables, which is particularly useful when not all attributes are mandatory for every record.

-

- **Example 1: Representing Missing Data**

- **Scenario:** A company maintains a database of employees. Some employees have not provided their middle names.

EmployeeID	FirstName	MiddleName	LastName
------------	-----------	------------	----------

1	John	NULL	Doe
2	Jane	Ann	Smith
3	Alice	NULL	Johnson

Example 2: Handling Inapplicable Data

Scenario: A university database tracks student enrollment. Some students are still enrolled, so their graduation date is not applicable.

StudentID	Name	EnrollmentDate	GraduationDate
-----------	------	----------------	----------------

101	John Smith	2020-09-01	NULL
102	Jane Doe	2019-09-01	2023-06-15

Example 3: Optional Fields in a Form

Scenario: An e-commerce website collects customer information. The "Phone Number" field is optional.

CustomerID	Name	Email	PhoneNumber
------------	------	-------	-------------

1	Alice Brown	alice@example.com	NULL
2	Bob Smith	bob@example.com	123-456-7890

Example 4: Aggregating Data with Null Values

Scenario: A sales database tracks the commission earned by sales representatives. Some representatives have not earned any commission yet.

SalesRepID	Name	Commission
1	John Doe	500.00
2	Jane Smith	NULL
3	Alice Brown	300.00

constraints (Domain, Key constraints, integrity constraints) and their importance

Constraints in a database management system (DBMS) are rules that ensure the data in a database is accurate, consistent, and reliable. They prevent errors and anomalies from entering the data. Here's an explanation of **Domain Constraints**, **Key Constraints**, and **Integrity Constraints**, along with examples:

Domain Constraints

- **Definition:** Domain constraints define the valid range of values that a particular attribute can have. They restrict the type and range of data that can be stored in a column, ensuring data integrity and consistency.
- **Example:** In a "Students" table, the "Age" column might have a domain constraint that only allows positive integers, ensuring that no student's age is recorded as negative or non-numeric.

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Age INT CHECK (Age > 0)  
);
```

Key Constraints

- **Definition:** Key constraints are used to uniquely identify records in a table. They include primary keys and unique keys.
 - **Primary Key:** A primary key is a column or set of columns that uniquely identifies each row in a table. It cannot contain null values.
 - **Unique Key:** A unique key ensures that all values in a column are different. Unlike a primary key, it can contain one null value.
- **Examples:**
 - **Primary Key:** In a "Customers" table, the "CustomerID" could be the primary key, ensuring each customer has a unique identifier.

```
CREATE TABLE Customers (  
    CustomerID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Email VARCHAR(100)  
);
```

Unique Key: In a "Users" table, the "Email" column might have a unique constraint to prevent duplicate email addresses.

```
CREATE TABLE Users (  
    UserID INT PRIMARY KEY,  
    Name VARCHAR(10),  
    Email VARCHAR(20) UNIQUE  
);
```

Integrity Constraints

Integrity constraints ensure that the data in a database remains consistent and accurate. They include:

- **Entity Integrity Constraints:** The **entity integrity constraint** states that no primary key value can be NULL.

- **Referential Integrity Constraints:** Ensure that relationships between tables are maintained by requiring that foreign keys in one table match primary keys in another table.
- **Check Constraints:** Specify conditions that data must meet before it can be entered into a table.

Examples of Integrity Constraints

- **Entity Integrity Constraint:** In an "Employees" table, the "EmployeeID" must be unique and non-null to ensure each employee can be identified uniquely.
- CREATE TABLE Employees (
- EmployeeID INT PRIMARY KEY,
- Name VARCHAR(100),
- Salary DECIMAL(10,2)
-);
- **Referential Integrity Constraint:** In a university database, if a student's course ID is a foreign key referencing the course ID in a "Courses" table, referential integrity ensures that only valid course IDs are entered for students.
- CREATE TABLE Courses (
- CourseID INT PRIMARY KEY,
- CourseName VARCHAR(100)
-);
-
- CREATE TABLE Students (
- StudentID INT PRIMARY KEY,
- Name VARCHAR(100),
- CourseID INT,
- FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)
-);

- **Check Constraint:** In a "Products" table, a check constraint might ensure that all product prices are greater than zero.

```
CREATE TABLE Products (  
    ProductID INT PRIMARY KEY,  
    ProductName VARCHAR(100),  
    Price DECIMAL(10,2) CHECK (Price > 0)  
);
```

Importance of Constraints

Constraints are crucial for maintaining data integrity and preventing inconsistencies:

- **Prevent Data Anomalies:** Constraints ensure that data is consistent and accurate, preventing errors such as duplicate records or invalid data types.
- **Maintain Relationships:** Referential integrity constraints ensure that relationships between tables are consistent, preventing orphaned records.
- **Improve Data Security:** By enforcing specific rules, constraints can help protect sensitive data by limiting the types of data that can be entered into certain fields.
- **Enhance Data Retrieval:** Constraints like primary keys improve data retrieval efficiency by providing unique identifiers for each record.

Relational Algebra

Relational Algebra consists of a set of operations that take one or more relations (tables) as input and produce a new relation as output. These operations are used to query and manipulate data in a relational database.

Basic Operations in Relational Algebra

1. Selection (σ)

- Selects tuples (rows) from a relation that satisfy a given condition.
- Syntax: $\sigma_{\langle \text{condition} \rangle}(R)$
- Example: $\sigma_{\text{age} > 30}(\text{Employee})$

- This selects all rows from the Employee table where the age is greater than 30.

2. Projection (π)

- Selects specific attributes (columns) from a relation.
- Syntax: $\pi_{\langle \text{attribute_list} \rangle}(R)$
- Example: $\pi_{\langle \text{name, age} \rangle}(\text{Employee})$
 - This selects only the name and age columns from the Employee table.

3. Union ($\cup$)

- Combines tuples from two relations, removing duplicates.
- Both relations must have the same set of attributes.
- Syntax: $R \cup S$
- Example: $\text{Employee} \cup \text{Manager}$
 - This combines rows from the Employee and Manager tables, removing any duplicates.

4. Set Difference ($-$)

- Finds tuples that are in one relation but not in another.
- Both relations must have the same set of attributes.
- Syntax: $R - S$
- Example: $\text{Employee} - \text{Manager}$
 - This finds rows that are in the Employee table but not in the Manager table.

5. Cartesian Product ($\times$)

- Combines each tuple from one relation with each tuple from another relation.
- Syntax: $R \times S$
- Example: $\text{Employee} \times \text{Department}$
 - This combines each row from the Employee table with each row from the Department table, resulting in a large table with all possible combinations.

6. Rename (ρ)

- Renames a relation or its attributes.
- Syntax: $\rho_{\langle \text{new_name} \rangle}(R)$ or $\rho_{\langle \text{new_name}(\text{attribute_list}) \rangle}(R)$
- Example: $\rho_{\langle \text{Emp}(\text{emp_id, name}) \rangle}(\text{Employee})$
 - This renames the Employee table to Emp and its attributes to emp_id and name.

Derived Operations

1. Intersection ($\cap$)

- Finds tuples that are common to both relations.
- Both relations must have the same set of attributes.
- Syntax: $R \cap S$
- Example: $\text{Employee} \cap \text{Manager}$
 - This finds rows that are present in both the Employee and Manager tables.

2. Join ($\bowtie$)

- Combines tuples from two relations based on a related column between them.
- Syntax: $R \bowtie_{\text{condition}} S$
- Example: $\text{Employee} \bowtie_{\text{Employee.dept_id} = \text{Department.dept_id}} \text{Department}$
 - This combines rows from the Employee and Department tables where the dept_id matches.

3. Division ($\div$)

- Finds tuples in one relation that match all tuples in another relation.
- Syntax: $R \div S$
- Example: $\text{Employee} \div \text{Department}$
 - This finds employees who work in all departments listed in the Department table.

Example

Consider the following tables:

Employee

emp_id	name	age	dept_id
1	Alice	25	101
2	Bob	30	102
3	Charlie	35	101

Department

dept_id	dept_name
101	HR
102	IT

1. Selection (σ)

- $\sigma_{\text{age} > 30}(\text{Employee})$

- Result:

emp_id	name	age	dept_id
3	Charlie	35	101

2. Projection (π)

- $\pi_{\text{name, age}}(\text{Employee})$

- Result:

name	age
Alice	25
Bob	30
Charlie	35

3. Join (⋈):

Student

student_id	name	age	dept_id
1	Alice	20	101
2	Bob	22	102
3	Charlie	21	103

Department

dept_id	dept_name
101	Computer
102	Mathematics
104	Physics

- Student ⋈_{Student.dept_id=Department.dept_id} Department
 - Result:

student_id	name	age	dept_id	dept_name
1	Alice	20	101	Computer
2	Bob	22	102	Mathematics

4. Union (U)

The **Union** operation combines rows from two relations and removes duplicates. Both relations must have the same schema (same number of attributes with matching domains).

Let

Manager

emp_id	name	age	dept_id
2	Bob	30	102
4	David	40	103

Employee U Manager

Result:

emp_id	name	age	dept_id
1	Alice	25	101
2	Bob	30	102
3	Charlie	35	101
4	David	40	103

5. Intersection ($\cap$)

The **Intersection** operation returns only the rows that are common to both relations. Both relations must have the same schema.

Example

Employee $\cap$ Manager

Result:

emp_id	name	age	dept_id
2	Bob	30	102

6. Cartesian Product ($\times$)

The **Cartesian Product** combines each row from the first relation with every row from the second relation. The result is a new relation with all possible combinations of rows from the two input relations.

Employee $\times$ Department

Result:

emp_id	name	age	dept_id	dept_id	dept_name
1	Alice	25	101	101	HR
1	Alice	25	101	102	IT
1	Alice	25	101	103	Finance
2	Bob	30	102	101	HR
2	Bob	30	102	102	IT
2	Bob	30	102	103	Finance
3	Charlie	35	101	101	HR
3	Charlie	35	101	102	IT
3	Charlie	35	101	103	Finance

Division in Relational Algebra

Division in relational algebra is a binary operation used to find tuples in one relation that are associated with **all** tuples in another relation. It is particularly useful for queries that involve the concept of "for all" or "all possible."

Notation:

- $R \div S$

R (Student, Course):

```
| Student | Course |
|-----|-----|
| Alice   | Math   |
| Alice   | Science|
| Bob     | Math   |
| Bob     | History|
| Charlie | Math   |
| Charlie | Science|
```

- S (Course):

Course
Math
Science

- $R \div S$

Student
Alice
Charlie

SYLLABUS

UNIT III: BASIC SQL: Simple Database schema, data types, table definitions (create, alter), different DML operations (insert, delete, update), Basic SQL querying (select and project) using where clause, arithmetic & logical operations, SQL functions (Date and Time, Numeric, String conversion). Creating tables with relationship, implementation of key and integrity constraints, nested queries, sub queries, grouping, aggregation, ordering, implementation of different types of joins, view (updatable and non-updatable), relational set operations.

Simple Database Schema

Definition

A database schema includes various objects such as tables, views, indexes, and procedures. Each schema is owned by a user and can be used to manage permissions and organize data logically.

Example

For instance, consider a simple schema for a bookstore:

```
CREATE SCHEMA bookstore;

CREATE TABLE bookstore.books
(
    id SERIAL PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    author VARCHAR(255) NOT NULL,
    publication_date DATE,
    price DECIMAL(10, 2) NOT NULL
);
```

In this example:

- A schema named bookstore is created.
- A table named books is defined within that schema with columns for id, title, author, publication_date, and price.

components of an SQL schema

The main components of an SQL schema are essential for defining how data is organized and managed within a database. Here are the key components:

1. Tables

Tables are the fundamental building blocks of a database schema. They consist of rows and columns, where:

- **Rows** represent individual records.
- **Columns** define the attributes or fields of those records.

For example, a books table might include columns for id, title, author, and price.

2. Data Types

Each column in a table has a specific data type that determines what kind of data can be stored in that column. Common data types include:

- **INT**: Integer values.
- **VARCHAR(n)**: Variable-length strings.
- **DATE**: Date values.
- **DECIMAL(M,D)**: Fixed-point numbers with specified precision.

3. Keys

Keys are crucial for maintaining data integrity and establishing relationships between tables:

- **Primary Key**: A unique identifier for each record in a table (e.g., id in the books table).
- **Foreign Key**: A field that links to the primary key of another table, establishing a relationship between the two tables.

4. Constraints

Constraints enforce rules on the data in the tables to maintain integrity. Common constraints include:

- **NOT NULL**: Ensures that a column cannot have NULL values.
- **UNIQUE**: Ensures all values in a column are different.
- **CHECK**: Validates that values meet specific conditions.

5. Views

Views are virtual tables created by querying data from one or more tables. They simplify complex queries and provide a way to present data without altering the underlying tables.

6. Indexes

Indexes improve the speed of data retrieval operations on a database table by providing quick access paths to rows based on indexed columns.

7. Stored Procedures and Functions

Stored procedures are precompiled SQL statements that can be executed as needed, while functions return values and can be used within SQL expressions.

8. Triggers

Triggers are special types of stored procedures that automatically execute in response to certain events on a particular table, such as insertions, updates, or deletions.

Data Types in SQL

SQL supports various data types to define the nature of data that can be stored in each column of a table. These data types can be broadly categorized into several groups:

1. Numeric Data Types

- **INT**: A standard integer type.
- **DECIMAL(M,D)**: A fixed-point number where *MM* is the total number of digits and *DD* is the number of digits after the decimal point.
- **FLOAT**: A floating-point number.

2. Character Data Types

- **CHAR(n)**: A fixed-length string.
- **VARCHAR(n)**: A variable-length string with a maximum length of *nn*.
- **TEXT**: A string with a maximum length of 65,535 characters.

3. Date and Time Data Types

- **DATE**: Stores dates in YYYY-MM-DD format.
- **DATETIME**: Combines date and time in YYYY-MM-DD HH:MM:SS format.
- **TIMESTAMP**: Stores a timestamp.

4. Binary Data Types

- **BINARY(n)**: Fixed-length binary data.
- **VARBINARY(n)**: Variable-length binary data.

Example of Data Types Usage

Here's how you might define different columns in a table using various data types:

```
CREATE TABLE employee
(
    employee_id INT PRIMARY KEY,
    name VARCHAR(100),
    salary DECIMAL(10, 2),
    hire_date DATE,
    profile_picture VARBINARY(MAX)
);
```

In this example:

- `employee_id` is an integer that uniquely identifies each employee.
- `name` is a variable-length string for storing employee names.

- salary uses the DECIMAL type for precise monetary values.
- hire_date stores the date when the employee was hired.
- profile_picture can store large binary files like images.

DML Operations: INSERT, DELETE, UPDATE

DML (Data Manipulation Language) operations are used to manage data within database tables. The three primary DML operations are INSERT, DELETE, and UPDATE. Let's explore each with examples.

1. INSERT Operation

The INSERT statement adds new rows to a table.

```
INSERT INTO table_name (column1, column2, ...)
```

```
VALUES (value1, value2, ...);
```

Examples:

Inserting a single row:

```
INSERT INTO employees (emp_id, name, department, salary)
```

```
VALUES (101, 'John Smith', 'Marketing', 50000);
```

Inserting multiple rows at once:

```
INSERT INTO employees (emp_id, name, department, salary)
```

```
VALUES
```

```
(102, 'Sarah Johnson', 'Sales', 55000),
```

```
(103, 'Michael Brown', 'IT', 60000),
```

```
(104, 'Emily Davis', 'HR', 48000);
```

Inserting without specifying columns (must provide values for all columns in order):

```
INSERT INTO employees
```

```
VALUES (105, 'Robert Wilson', 'Finance', 65000);
```

2. DELETE Operation

The DELETE statement removes rows from a table.

```
DELETE FROM table_name
```

```
WHERE condition;
```

Examples:

Deleting specific rows:

```
DELETE FROM employees
```

```
WHERE emp_id = 104;
```

Deleting with multiple conditions:

```
DELETE FROM employees
```

```
WHERE department = 'Sales' AND salary < 50000;
```

Deleting all rows (be careful!):

```
DELETE FROM employees;
```

```
-- This removes all data but keeps the table structure
```

3. UPDATE Operation

The UPDATE statement modifies existing data in a table.

Basic Syntax:

```
UPDATE table_name
```

```
SET column1 = value1, column2 = value2, ...
```

```
WHERE condition;
```

Examples:

Updating a single column:

```
UPDATE employees
```

```
SET salary = 52000
```

```
WHERE emp_id = 101;
```

Updating multiple columns:

```
sql
```

```
UPDATE employees
```

```
SET department = 'Operations', salary = salary * 1.1
```

```
WHERE department = 'IT';
```

Updating with calculations:

```
sql
```

```
UPDATE employees
```

```
SET salary = salary * 1.05
```

```
WHERE hire_date < '2020-01-01';
```

Basic SQL Querying (SELECT and PROJECT)

SELECT with WHERE Clause

Retrieve specific columns (projection) and filter rows (selection):

sql

Copy

```
-- Select all columns from the 'employees' table where department is 'IT'
SELECT * FROM employees WHERE department = 'IT';

-- Project specific columns with a condition
SELECT name, salary FROM employees WHERE salary > 50000;
```

Arithmetic & Logical Operations

sql

Copy

```
-- Calculate annual salary from monthly salary
SELECT name, monthly_salary * 12 AS annual_salary FROM employees;

-- Combine arithmetic and logical operators
SELECT name, department
FROM employees
WHERE salary >= 50000 AND (department = 'Sales' OR department = 'Marketing');
```

UNIT 4

Syllabus:

Introduction to PL/SQL programming – Advantages of PL/SQL, The generic PL/SQL block, Character set, Literals, Data types, Variables, Constants, Comments, Conditional and Control statements, Iterative statements, Exception handling, Procedures, Functions, Cursors, Triggers. **Schema Refinement (Normalization):** Purpose of Normalization or schema refinement, concept of functional dependency, normal forms based on functional dependency Lossless join and dependency preserving decomposition, (1NF, 2NF and 3 NF), concept of surrogate key, Boyce-Codd normal form (BCNF), MVD, Fourth normal form(4NF), Fifth Normal Form (5NF). 3102960252552401

Introduction to PL/SQL Programming

PL/SQL (Procedural Language/Structured Query Language) is Oracle Corporation's procedural extension to SQL. It allows us to write full-fledged programs to interact with an Oracle database, incorporating loops, conditionals, functions, procedures, and exception handling — something basic SQL alone cannot do.

It combines the power of SQL with the procedural features of programming languages like C or Java.

Basic PL/SQL Structure:

DECLARE

-- Variable declarations

BEGIN

-- SQL and procedural statements

EXCEPTION

-- Exception handling

END;

Advantages of PL/SQL:

Advantage	Explanation	Real-time Example
1. Block-structured Language	Code is organized into blocks, making it modular, readable, and maintainable.	A bank application uses procedures like Process_Withdrawal, Process_Deposit, and Calculate_Interest, each encapsulated in separate blocks.
2. Integration with SQL	PL/SQL allows embedding SQL directly in procedural code.	An HR system updates employee salaries using UPDATE statements within a loop over departments.

Advantage	Explanation	Real-time Example
3. Performance Improvement	Multiple SQL statements can be sent to the database in one block, reducing network traffic.	A retail app updates inventory for 1,000 items using a single PL/SQL block instead of 1,000 separate SQL calls.
4. Error Handling (Exception Handling)	PL/SQL provides structured exception handling to catch and respond to run-time errors.	In a money transfer app, if the debit from the sender fails, PL/SQL rolls back and logs the error without deducting anything.
5. Portability	PL/SQL code is portable across Oracle environments.	A PL/SQL function that calculates tax can be reused across Oracle-based apps in finance and billing.
6. Support for Cursors	Cursors in PL/SQL allow row-by-row processing of SQL result sets.	An e-commerce system processes orders one by one from a Pending_Orders table using an explicit cursor.
7. Support for Stored Procedures and Functions	Code can be stored in the database and reused.	A hospital system has a stored function Calculate_BMI(weight, height) used in various forms and reports.
8. Triggers for Automation	PL/SQL allows creation of triggers that automatically execute in response to events.	In a university database, a trigger logs changes to student grades into an audit table.

Example1 : Salary Update

```

BEGIN
    UPDATE employees
    SET salary = salary * 1.10
    WHERE department_id = 10;
    DBMS_OUTPUT.PUT_LINE('Salary updated successfully for department 10.');
```

EXCEPTION

```

    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
```

END;

/

Example 2:

```

BEGIN
    DBMS_OUTPUT.PUT_LINE('Hello World');
```

```
END;
```

```
/
```

2. Character Set

The character set in Oracle defines the encoding scheme for storing and processing text data.

```
DECLARE
```

```
    v_name VARCHAR2(50 CHAR); -- CHAR specifies character length (not bytes)
```

```
BEGIN
```

```
    v_name := 'Hare Ram Singh';
```

```
    DBMS_OUTPUT.PUT_LINE('Name: ' || v_name);
```

```
END;
```

```
/
```

3. Literals

Literals are fixed values in PL/SQL code, including **numeric literals**, **character literals**, **string literals**, **boolean literals**, and **date/time literals**.

-

```
DECLARE
```

```
    v_number NUMBER := 42; -- Numeric literal
```

```
    v_string VARCHAR2(20) := 'Hello'; -- String literal
```

```
    v_boolean BOOLEAN := TRUE; -- Boolean literal
```

```
    v_date DATE := TO_DATE('2025-05-04', 'YYYY-MM-DD'); -- Date literal
```

```
BEGIN
```

```
    DBMS_OUTPUT.PUT_LINE('Number: ' || v_number);
```

```
    DBMS_OUTPUT.PUT_LINE('String: ' || v_string);
```

```
    IF v_boolean THEN
```

```
        DBMS_OUTPUT.PUT_LINE('Boolean is TRUE');
```

```
    END IF;
```

```
    DBMS_OUTPUT.PUT_LINE('Date: ' || TO_CHAR(v_date, 'YYYY-MM-DD'));
```

```
END;
```

```
/
```

4. Data Types

PL/SQL supports various data types, including **Scalar** (e.g., NUMBER, VARCHAR2, DATE, BOOLEAN), **Composite** (e.g., RECORD, TABLE), **Reference** (e.g., REF CURSOR), and **LOB** (e.g., CLOB, BLOB).

DECLARE

```
v_salary NUMBER(10,2) := 50000.75; -- Scalar: NUMBER  
v_name VARCHAR2(100) := 'John Doe'; -- Scalar: VARCHAR2  
v_hire_date DATE := SYSDATE; -- Scalar: DATE  
TYPE t_address IS RECORD ( -- Composite: RECORD  
    street VARCHAR2(50),  
    city VARCHAR2(50)  
);  
v_address t_address;
```

BEGIN

```
v_address.street := '123 Main St';  
v_address.city := 'New York';  
DBMS_OUTPUT.PUT_LINE('Name: ' || v_name || ', Salary: ' || v_salary);  
DBMS_OUTPUT.PUT_LINE('Address: ' || v_address.street || ', ' || v_address.city);  
DBMS_OUTPUT.PUT_LINE('Hire Date: ' || TO_CHAR(v_hire_date, 'YYYY-MM-DD'));
```

END;

/

5. Variables

Variables are named storage locations in PL/SQL to hold data. They are declared in the DECLARE section with a specific data type.

DECLARE

```
v_employee_id NUMBER := 100;  
v_employee_name VARCHAR2(50) := 'Alice Smith';  
v_salary NUMBER(10,2);
```

BEGIN

```
v_salary := 60000.50;  
DBMS_OUTPUT.PUT_LINE('Employee ID: ' || v_employee_id);  
DBMS_OUTPUT.PUT_LINE('Name: ' || v_employee_name);
```

```

    DBMS_OUTPUT.PUT_LINE('Salary: ' || v_salary);
END;

/

```

6. Constants

Constants are variables whose values cannot be changed after initialization. They are declared using the CONSTANT keyword.

```

DECLARE
    c_pi CONSTANT NUMBER := 3.14159;
    v_radius NUMBER := 5;
    v_area NUMBER;
BEGIN
    v_area := c_pi * v_radius * v_radius;
    DBMS_OUTPUT.PUT_LINE('Area of circle: ' || v_area);
END;

/

```

7. Comments

Comments explain code and are ignored by the PL/SQL compiler. Single-line comments use --, and multi-line comments use /* */.

```

DECLARE
    v_counter NUMBER := 0; -- Initialize counter
BEGIN
    /* This block increments the counter
       and prints its value */
    v_counter := v_counter + 1;
    DBMS_OUTPUT.PUT_LINE('Counter: ' || v_counter); -- Display counter
END;

/

```

8. Conditional and Control Statements

PL/SQL supports conditional statements like IF-THEN-ELSE and CASE to control program flow.

```

DECLARE
    v_salary NUMBER := 45000;
BEGIN
    IF v_salary > 50000 THEN
        DBMS_OUTPUT.PUT_LINE('High Salary');
    ELSIF v_salary BETWEEN 30000 AND 50000 THEN
        DBMS_OUTPUT.PUT_LINE('Medium Salary');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Low Salary');
    END IF;
END;

/

```

9. Iterative Statements

PL/SQL provides loops like FOR, WHILE, and LOOP for iterative execution.

```
DECLARE
    v_sum NUMBER := 0;
BEGIN
    FOR i IN 1..5 LOOP
        v_sum := v_sum + i;
        DBMS_OUTPUT.PUT_LINE('Iteration ' || i || ': Sum = ' || v_sum);
    END LOOP;
END;
/
```

10. Exception Handling

Exception handling manages runtime errors using the EXCEPTION section, with predefined (e.g., NO_DATA_FOUND) or user-defined exceptions.

```
DECLARE
    v_employee_name VARCHAR2(50);
BEGIN
    SELECT first_name INTO v_employee_name
    FROM employees
    WHERE employee_id = 999; -- Non-existent ID
    DBMS_OUTPUT.PUT_LINE('Employee: ' || v_employee_name);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: No employee found!');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Unexpected error: ' || SQLERRM);
END;
/
```

11. Procedures

Procedures are stored PL/SQL subprograms that perform specific tasks and can accept parameters but do not return values.

Example:

```
CREATE OR REPLACE PROCEDURE update_salary(p_employee_id IN NUMBER, p_increase IN
NUMBER) AS
BEGIN
    UPDATE employees
    SET salary = salary + p_increase
    WHERE employee_id = p_employee_id;
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Salary updated for Employee ID: ' || p_employee_id);
```

```

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Employee ID not found!');
END;
/
-- Execute the procedure
EXEC update_salary(100, 500);

```

12. Functions

Functions are stored PL/SQL subprograms that return a single value and can be used in SQL statements.

```

CREATE OR REPLACE FUNCTION calculate_bonus(p_salary IN NUMBER) RETURN NUMBER AS
  v_bonus NUMBER;
BEGIN
  v_bonus := p_salary * 0.1; -- 10% bonus
  RETURN v_bonus;
END;
/
-- Use the function
DECLARE
  v_salary NUMBER := 60000;
  v_bonus NUMBER;
BEGIN
  v_bonus := calculate_bonus(v_salary);
  DBMS_OUTPUT.PUT_LINE('Bonus for salary ' || v_salary || ': ' || v_bonus);
END;
/

```

13. Cursors

Cursors retrieve and process multiple rows from a query result set. They can be **implicit** (managed by PL/SQL) or **explicit** (user-defined).

```

DECLARE
  CURSOR emp_cursor IS
    SELECT first_name, salary
    FROM employees
    WHERE department_id = 10;
  v_name employees.first_name%TYPE;
  v_salary employees.salary%TYPE;
BEGIN
  OPEN emp_cursor;
  LOOP
    FETCH emp_cursor INTO v_name, v_salary;
    EXIT WHEN emp_cursor%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE('Name: ' || v_name || ', Salary: ' || v_salary);
  END LOOP;
  CLOSE emp_cursor;

```

```
END;  
/
```

14. Triggers

Triggers are PL/SQL blocks automatically executed in response to specific database events (e.g., INSERT, UPDATE, DELETE).

```
CREATE TABLE salary_audit (  
  employee_id NUMBER,  
  old_salary  NUMBER,  
  new_salary  NUMBER,  
  update_date DATE  
);  
  
CREATE OR REPLACE TRIGGER log_salary_update  
BEFORE UPDATE OF salary ON employees  
FOR EACH ROW  
BEGIN  
  INSERT INTO salary_audit (employee_id, old_salary, new_salary, update_date)  
  VALUES (:OLD.employee_id, :OLD.salary, :NEW.salary, SYSDATE);  
  DBMS_OUTPUT.PUT_LINE('Salary updated for Employee ID: ' || :OLD.employee_id);  
END;  
/  
-- Test the trigger  
UPDATE employees  
SET salary = salary + 1000  
WHERE employee_id = 100;
```

Purpose of Normalization (Schema Refinement)

What is Normalization?

Normalization is the process of **organizing data in a database** to:

- Eliminate **data redundancy** (repetition of data).
- Ensure **data integrity**.
- Simplify **data maintenance and updates**.

Purpose:

- **Avoid anomalies** (insertion, deletion, update).
- **Efficient storage**.
- **Better design** using smaller, related tables.

2. Functional Dependency (FD)

Definition:

A **functional dependency (FD)** is a relationship between attributes in a relation.

If $A \rightarrow B$, it means:

"Attribute A *functionally determines* attribute B"

(i.e., if you know A, you can find exactly one value of B)

Roll_No	Name	Dept
101	Alice	CSE
102	Bob	ECE

- Roll_No → Name ✓ (Each Roll_No has exactly one Name)
- Roll_No → Dept ✓

But:

- Name → Roll_No ✗ (Name is not always unique)

Normal Forms Based on Functional Dependency

Normalization is done in **steps** called **normal forms** (NFs). Each form fixes certain types of redundancy.

◆ 1NF (First Normal Form):

Rule: No **repeating groups** or arrays; each cell must hold a **single value**.

✗ Bad (Not in 1NF):

Roll_No	Name	Subjects
101	John	DBMS, OS, CN

Roll_No	Name	Subject
101	John	DBMS
101	John	OS
101	John	CN

2NF (Second Normal Form):

Rule:

- Must be in 1NF.
- No **partial dependency** on a **composite key**.

Partial Dependency: A non-key attribute depends on part of the composite primary key.

StudentID	CourseID	CourseName
1	DB101	DBMS

Here:

- Composite key = (StudentID, CourseID)
- CourseName depends **only on CourseID**, not on the full key → **Partial Dependency**

-  **Solution:**
- Break into 2 tables:
- **StudentCourse Table:**

StudentID	CourseID
1	DB101

Course Table:

CourseID	CourseName
DB101	DBMS

◆ 3NF (Third Normal Form):

Rule:

- Must be in 2NF.
- No **transitive dependency** ($A \rightarrow B \rightarrow C$, then $A \rightarrow C$ is indirect)

 **Example (Not in 3NF):**

EmpID	EmpName	DeptID	DeptName
1	Alice	10	CSE

Here:

- $\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}$
- $\text{EmpID} \rightarrow \text{DeptName}$ is transitive

 **Solution:**

Break into:

Employee Table:

EmpID	EmpName	DeptID
1	Alice	10

Department Table:

DeptID	DeptName
10	CSE

◆ BCNF (Boyce-Codd Normal Form):

What is Trivial Functional Dependency?

A functional dependency is called **trivial** if the **right-hand side (RHS)** is a **subset of the left-hand side (LHS)**.

Example:

- $\{A, B\} \rightarrow A$ is **trivial**, because A is part of the LHS $\{A, B\}$.
- $\{X, Y\} \rightarrow X$ or $\{X\} \rightarrow X$ are also trivial.

◆ Non-Trivial Functional Dependency

A **non-trivial functional dependency** is one where the **RHS is not a subset of the LHS**.

Example:

- $\{A\} \rightarrow B$ is **non-trivial**, if B is not part of $\{A\}$.
- $\{A, B\} \rightarrow C$ is non-trivial if C is not in $\{A, B\}$.

✓ Boyce-Codd Normal Form (BCNF)

BCNF is a **stronger version of 3rd Normal Form (3NF)**. A relation is in BCNF **if and only if**:

For every non-trivial functional dependency $X \rightarrow Y$,

X must be a super key of the relation.

Consider a table:

Student	Teacher	Subject
Jhansi	P.Naresh	Database
Jhansi	K.Das	C
Subbu	P.Naresh	Database
Subbu	R.Prasad	C

Functional Dependencies:

- $(\text{Student}, \text{Teacher}) \rightarrow \text{Subject}$
- $(\text{Student}, \text{Subject}) \rightarrow \text{Teacher}$
- $\text{Teacher} \rightarrow \text{Subject}$

But is it in BCNF?

- The functional dependency $\text{Teacher} \rightarrow \text{Subject}$ violates BCNF because Teacher is not a candidate key¹².
- This means a teacher can teach only one subject, but a subject can be taught by multiple teachers.

BCNF Decomposition

To bring the table into BCNF, decompose it based on the violating dependency ($\text{Teacher} \rightarrow \text{Subject}$):

Table 1: Teacher_Subject

Teacher	Subject
P.Naresh	Database
K.Das	C
R.Prasad	C

Table 2: Student_Teacher

Student	Teacher
Jhansi	P.Naresh
Jhansi	K.Das
Subbu	P.Naresh
Subbu	R.Prasad

Now, both tables are in BCNF:

- In Teacher_Subject, Teacher is the key.
- In Student_Teacher, (Student, Teacher) is the key¹.

Summary Table: 3NF vs BCNF

Normal Form	Requirement
3NF	For every FD $X \rightarrow YX \rightarrow Y$, either XX is a super key, or YY is a prime attribute
BCNF	For every FD $X \rightarrow YX \rightarrow Y$, XX must be a super key

What is 4NF (Fourth Normal Form)?

A relation is in **4NF** if:

It is in **Boyce-Codd Normal Form (BCNF)**

and

it **does not have any multi-valued dependencies**.

What is a Multi-Valued Dependency (MVD)?

A **multi-valued dependency** occurs when:

One attribute in a table **independently determines multiple values** of another attribute, and these values are **not related** to other attributes in the row.

Simple Example (Violation of 4NF)

Let's say we have a table Student:

StudentID	Hobby	Language
101	Painting	English
101	Painting	Hindi
101	Singing	English
101	Singing	Hindi

This table tells us:

- Student 101 has **two hobbies**: Painting and Singing
- Student 101 knows **two languages**: English and Hindi

But **Hobbies and Languages are unrelated** — they're both independently dependent on StudentID.

Problem:

There is a **multi-valued dependency**:

- StudentID $\twoheadrightarrow$ Hobby
- StudentID $\twoheadrightarrow$ Language

So the table has **redundant data** — a Cartesian product between hobbies and languages.

How to Convert to 4NF:

Break it into two separate tables:

Table 1: StudentHobby

StudentID	Hobby
101	Painting
101	Singing

Table 2: StudentLanguage

StudentID	Language
101	English
101	Hindi

☑ No multi-valued dependencies exist in either table.

☑ Both are in **4NF**.

Fifth Normal Form(5NF):

- 5NF deals with *join dependencies*, which occur when a table can be split into three or more tables and then losslessly joined back together [26](#).
- 5NF ensures that decomposing a table into smaller tables does not result in spurious (incorrect) data when joined back

Simple Example

Suppose you have a table recording which **Agent** sells which **Product** for which **Company**:

Agent	Product	Company
Smith	Car	Ford
Smith	Truck	GM
Jones	Car	GM

The table says:

- Smith sells Cars for Ford.
- Smith sells Trucks for GM.
- Jones sells Cars for GM.

Decomposition to 5NF:

Break the table into three tables:

Agent-Product

Agent	Product
Smith	Car
Smith	Truck
Jones	Car

Agent-Company

Agent	Company
Smith	Ford
Smith	GM
Jones	GM

Product-Company:

Product	Company
Car	Ford
Car	GM
Truck	GM

Now, joining these three tables using natural joins gives you back the original information, but without redundancy or spurious data. No extra or missing combinations appear, and each fact is stored only once

Syllabus:

UNIT V: Transaction Concept: Transaction State, ACID properties, Concurrent Executions, Serializability, Recoverability, Implementation of Isolation, Testing for Serializability, lock based, time stamp based, optimistic, concurrency protocols, Deadlocks, Failure Classification, Storage, Recovery and Atomicity, Recovery algorithm.

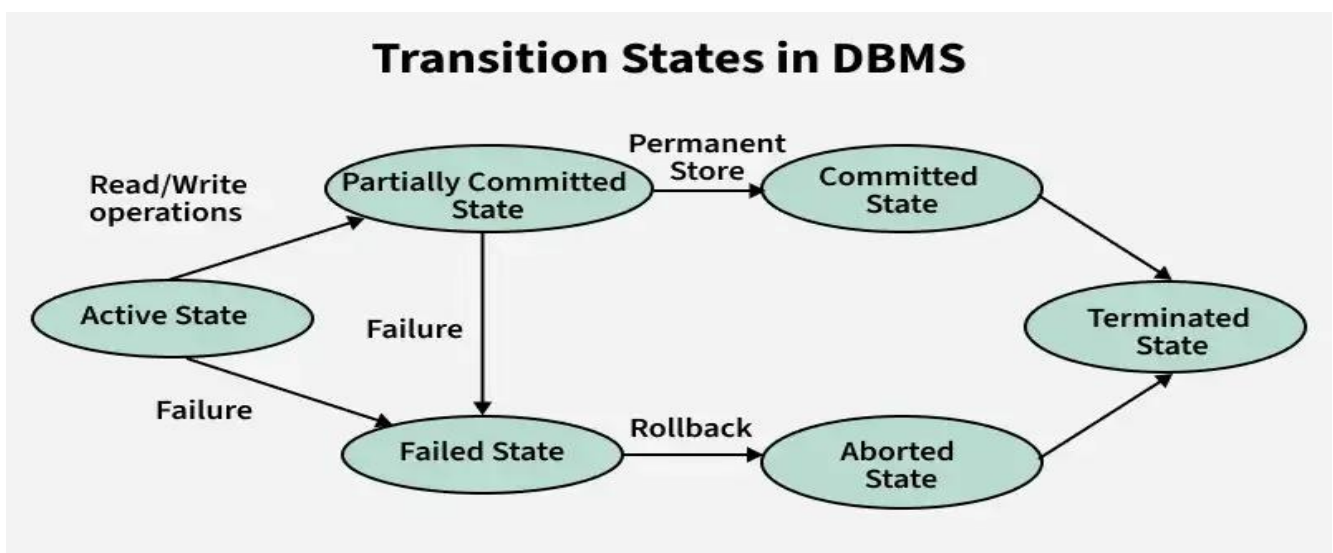
Transaction:

A transaction in a DBMS refers to a series of operations that are executed together as a single unit to perform a task, such as transferring money between accounts in a banking system.

Transaction States in DBMS

These are the different types of Transaction States:

1. Active State
2. Partially Committed State
3. Failed State
4. Aborted State
5. Committed State
6. Terminated State



State	Description	Example
Active	Transaction is executing operations.	BEGIN TRANSACTION; UPDATE accounts SET balance = balance - 100 WHERE user_id = 1;
Partially Committed	All operations are executed, but changes are not yet saved permanently.	After the last statement executes but before COMMIT.
Committed	Changes are permanently saved to the database.	COMMIT; → Now, the balance update is permanent.
Failed	Transaction cannot proceed (due to errors, constraints, etc.).	UPDATE accounts SET balance = balance - 1000 WHERE user_id = 1; (Fails if balance < 1000)
Aborted	Transaction is rolled back, and the database is restored to its previous state.	ROLLBACK; → Reverts all changes made in the transaction.
Terminated	Transaction is either committed or aborted and no longer active.	

ACID Properties:

1. Atomicity

Ensures all operations in a transaction complete successfully, or none do.

- **Example:** Transferring \$100 between bank accounts. If either the withdrawal or deposit fails, the entire transaction is rolled back

```
BEGIN TRANSACTION;
```

```
UPDATE accounts SET balance = balance - 100 WHERE user_id = 1; -- Debit
```

```
UPDATE accounts SET balance = balance + 100 WHERE user_id = 2; -- Credit
```

```
COMMIT;
```

- If the **credit** fails, the **debit is rolled back** (no partial updates).

(2) Consistency

- **Definition:** A transaction brings the database from one valid state to another (enforcing rules like constraints, triggers, etc.).
- **Example:**
 - If an accounts table has a constraint $\text{balance} \geq 0$, any transaction that makes balance negative will be aborted.

3. Isolation

Prevents concurrent transactions from interfering.

- **Example:** Two users attempting to buy the last item in stock. Isolation ensures only one transaction succeeds, updating inventory correctly

4. Durability

- **Definition:** Once a transaction is committed, its changes persist even after a system crash.
- **Example:**
 - After COMMIT, even if the database crashes, the balance updates remain intact when the system recovers.

Concurrent Executions

Concurrent execution means **multiple transactions are executed at the same time** (i.e., interleaved), rather than one after the other.

Why we need it:

- Improves performance (utilizes CPU & disk better)
- Increases throughput (multiple users can work simultaneously)
- Reduced average response time
-

Example:

Let's consider two transactions T1 and T2 on a **bank account** having a balance of ₹5000.

- T1: Withdraw ₹1000
- T2: Deposit ₹2000

Serial Execution (one after another):

T1: Read(balance = 5000)

balance = balance - 1000 = 4000

Write(balance = 4000)

T2: Read(balance = 4000)

balance = balance + 2000 = 6000

Write(balance = 6000)

Concurrent Execution (interleaved):

T1: Read(balance = 5000)

T2: Read(balance = 5000)

T1: balance = balance - 1000 = 4000

T1: Write(balance = 4000)

T2: balance = balance + 2000 = 7000 ← Incorrect!

T2: Write(balance = 7000)

Problem: Final balance should be ₹6000, but due to interleaving, it became ₹7000. This is a lost update problem.

Serializability

Serializability ensures that even though transactions are executed concurrently, the final result is equivalent to some serial execution of those transactions.

Types of Serializability:

1. **Conflict Serializability:** Based on non-conflicting operations (read/write rules).

Two operations conflict if:

- They belong to different transactions
- They operate on the same data item
- At least one of them is a write operation

2. **View Serializability:** Based on read-from relationships.

Example:

Let's say we have two transactions again:

- T1: Read(A), Write(A)
- T2: Read(A), Write(A)

Schedule (order of operations):

T1: Read(A)
T2: Read(A)
T1: Write(A)
T2: Write(A)

This is **not serializable**, because the final result is not the same as any serial execution (T1 then T2 or T2 then T1).

But this one **is serializable**:

T1: Read(A)
T1: Write(A)
T2: Read(A)
T2: Write(A)

It is equivalent to T1 followed by T2.

Goal: Allow concurrency **without violating correctness**.

Recoverability

Definition:

A schedule is **recoverable** if a transaction that **reads data written by another transaction** commits **only after the writer transaction commits**.

Why important?

To prevent using dirty (uncommitted) data which may be rolled back later.

Example:

Let's say:

- T1: Write(A)
- T2: Read(A), Commit
- T1: Rollback

This is **non-recoverable**, because T2 used data written by T1, but T1 rolled back. So now T2 is using invalid data.

Types:

1. **Recoverable Schedule:** Transactions commit only after all transactions they read from have committed
2. **Cascadeless Schedule:** Transactions can only read committed data (stronger than recoverable)

Recoverable schedule:

- T1: Write(A)
- T2: Read(A)
- T1: Commit
- T2: Commit

Now it's safe, because T2 reads only **committed** data.

Example of Non-Recoverable Schedule:

T1: Read(A)

T1: $A = A + 100$

T1: Write(A)

T2: Read(A) // T2 reads uncommitted data from T1

T1: Rollback // Now T2 has read data that doesn't exist

Example of Recoverable Schedule:

T1: Read(A)

T1: $A = A + 100$

T1: Write(A)

T2: Read(A) // T2 reads uncommitted data from T1

T1: Commit // T1 commits first

T2: Commit // Then T2 commits - this is recoverable

Example of Cascadeless Schedule:

T1: Read(A)

T1: $A = A + 100$

T1: Write(A)

T1: Commit

T2: Read(A) // Only reads committed data

T2: Commit

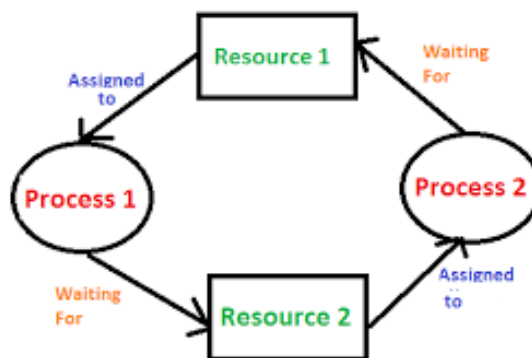
These concepts are fundamental to maintaining database consistency while allowing concurrent access to data.

Deadlocks in DBMS

A **deadlock** occurs when two or more transactions are waiting for each other to release locks on resources, and none of them can proceed.

Example:

- T1 locks Resource A and waits for Resource B.
- T2 locks Resource B and waits for Resource A.
Now both are waiting indefinitely.



Conditions for Deadlock:

- Mutual exclusion
- Hold and wait
- No preemption
- Circular wait

1. Mutual Exclusion:

A resource can only be used by one process at a time. If another process requests that resource, it must wait until the resource is released.

- **Example:**

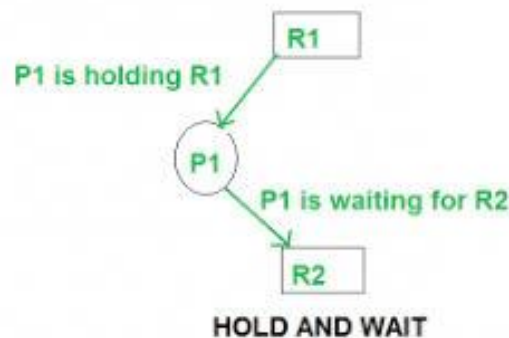
If a printer is in use by Process A, Process B must wait for Process A to finish using the printer before it can print.

2. Hold and Wait:

A process is holding at least one resource and is waiting to acquire another resource that is currently held by another process.

- **Example:**

Process P1 is holding resource R1 and is waiting for resource R2, which is being held by Process P2.



3. No Preemption:

Resources cannot be taken away from a process by force. A process must release resources voluntarily when it's finished using them.

- **Example:**

If Process A is holding resource R1, the operating system cannot take R1 away from Process A to give it to Process B.

4. Circular Wait:

A circular chain of processes exists where each process is waiting for a resource held by the next process in the chain.

- **Example:**

Process A is waiting for R1 held by B, B is waiting for R2 held by C, and C is waiting for R3 held by A.

Deadlock Handling Strategies:

- **Detection and Recovery:** The system allows deadlocks to occur but has mechanisms to detect and resolve them, typically by aborting one or more transactions to break the cycle.
- **Prevention:** The system proactively ensures that deadlocks do not occur by imposing certain constraints, such as ordering resource requests or using timeouts.

Deadlock Prevention: Two Main Approaches

Approach 1: Prevent Cyclic Waits (Resource Ordering)

1. All Locks at Once (Static Locking):

- A transaction must lock all its required data items before execution.
- Either all locks are acquired in one step, or none are.
- ● Disadvantages:
 1. Hard to predict all required data items in advance.
 2. Low data utilization – data items may be locked but unused for long periods.

2. Ordering of Data Items (Lock Ordering):

- All data items are assigned a total or partial order.
- Transactions must request locks in a sequence consistent with this order.
- Example: Once a transaction locks item X, it **cannot request locks on any item before X** in the order.

Approach 2: Preemption & Rollbacks (Timestamp-Based Techniques)

- When a potential deadlock is detected, instead of waiting, **a transaction may be rolled back** (preempted).
- Each transaction gets a **unique timestamp** used for decision making (not for locking itself).
- If a transaction is **rolled back**, it retains the **same timestamp** on restart.

Deadlock Detection Using Wait-For Graph (WFG)

Wait-For Graph (WFG):

- Definition: A directed graph $G = (V, E)$ where:
 - V: Set of vertices = all transactions in the system.
 - E: Set of directed edges; an edge $T_i \rightarrow T_j$ exists if T_i is waiting for T_j to release a data item.

Operations on the WFG:

- **Add edge** $T_i \rightarrow T_j$: When T_i requests a lock held by T_j .
- **Remove edge** $T_i \rightarrow T_j$: When T_j releases the data item and T_i is no longer waiting.

Deadlock Condition

- A **deadlock exists** *if and only if* the **wait-for graph contains a cycle**.
- All transactions in the cycle are **deadlocked**.

Example Scenario (from the book):

- Initial state (no cycle):

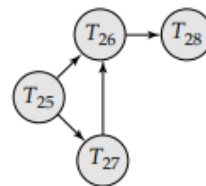


Figure 16.18 Wait-for graph with no cycle.

- $T_{25} \rightarrow T_{26}$, $T_{25} \rightarrow T_{27}$
- $T_{27} \rightarrow T_{26}$
- $T_{26} \rightarrow T_{28}$
→ No cycle → No deadlock
- Now, if $T_{28} \rightarrow T_{27}$ is added → **Cycle**:
 - **$T_{26} \rightarrow T_{28} \rightarrow T_{27} \rightarrow T_{26}$**
→ Deadlock among T26, T27, T28.

•

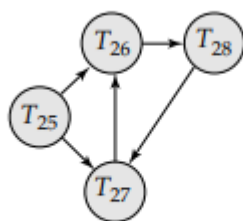


Figure 16.19 Wait-for graph with a cycle.

Recovery from Deadlock

Selection of a victim. Given a set of deadlocked transactions, we must determine which transaction (or transactions) to roll back to break the deadlock.

Rollback: We should roll back those transactions that will incur the minimum cost.

Once we have decided that a particular transaction must be rolled back, we must determine how far this transaction should be rolled back. Partial rollback or total rollback

Starvation.

In a system where the selection of victims is based primarily on cost factors, it may happen that the same transaction is always picked as a victim. As a result, this transaction never completes its designated task, thus there is starvation.